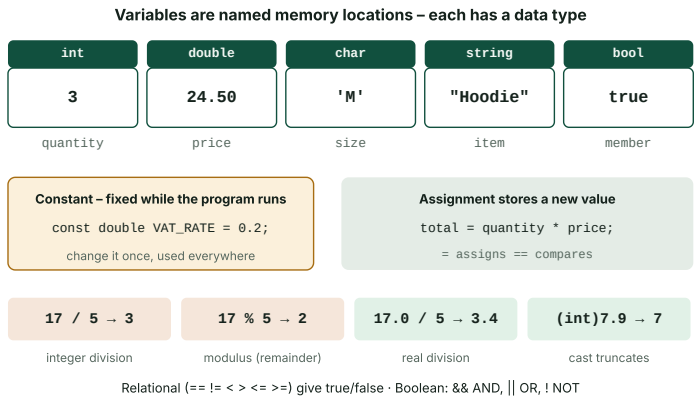
IDEs, translators, data types, variables and operators

AP19 FACT FILE

Specification: students should be able to

- Describe the main features of an integrated development environment (IDE), including the editor, preferences, compiler and debugging tools
- Explain the process of translation and the differences between assemblers, compilers and interpreters
- Explain the terms syntax, data type and variable, and use the data types integer, real, character, string and Boolean appropriately
- Explain the use of variables and constants, and declare, initialise and assign them in a high-level language
- Use arithmetic, relational and Boolean (logical) operators, including integer division and modulus, and evaluate conditions
- Write programs that take input from the user, convert it to a suitable type and produce formatted output

Key words: IDE, source code, object code, syntax, translator, assembler, compiler, interpreter, data type, variable, identifier, constant, assignment, casting, modulus, integer division, relational operator, Boolean operator



Variables as labelled memory boxes with their C# data types, a locked constant, and examples of integer division and modulus

Overview: from algorithm to running program

Once an algorithm has been designed, a programmer expresses it in a programming language such as C#. The result is source code: text that people can read and edit but that the processor cannot execute directly. Programmers usually work inside an integrated development environment (IDE), which combines an editor, a translator and debugging tools in one application so that code can be written, checked, run and corrected quickly. Before the program can run, its source code must be translated into machine code, the binary instructions the processor understands; this job is done by an assembler, a compiler or an interpreter, each working in a different way. Inside the program, data is held in variables. Each variable has an identifier and a data type, which decides what values it can hold, how much memory is reserved and which operations make sense. Values that must never change are declared as constants. Operators then do the work: arithmetic operators calculate, relational operators compare values to give true or false, and Boolean operators combine those conditions. Finally, programs communicate with users through input and output, converting text typed at the keyboard into numbers and formatting results neatly on screen. These ideas are the foundation for every program you will write in the AS course.

The integrated development environment

An IDE is a software application that gives programmers a complete set of tools for developing software. Its editor lets code be written, edited and saved, and most editors are smart: syntax highlighting colours keywords, strings and comments differently, automatic indentation shows which statements belong inside which blocks, and line numbers and code folding make long programs easier to navigate. A syntax checker underlines invalid code as it is typed, and code completion (called IntelliSense in Visual Studio) suggests keywords and identifiers after a few letters, improving speed and reducing spelling errors. Preferences allow a team to agree layout rules, such as whether an opening brace goes on its own line, and the IDE can apply them automatically so everyone's code looks the same when it is combined. The IDE also contains or connects to a compiler, so the program can be built and run with a single command, and any translation errors are listed with their line numbers. Finally, debugging tools allow the programmer to set breakpoints, step through code one line at a time and watch the values of variables, which makes locating errors far easier. Debugging techniques are covered in more detail in AP23.

Translators

Translation is the conversion of a program from the language it was written in into machine code that the processor can execute. An assembler translates assembly language, a low-level language in which each mnemonic, such as LDA or ADD, corresponds to exactly one machine code instruction. High-level languages are translated by compilers or interpreters. A compiler translates the whole program before any of it runs. If errors are found, it produces an error report and no executable is created until every error is corrected, after which the program must be compiled again. Because the compiler sees the complete program, it can optimise the object code, so the finished program runs quickly and can be distributed without its source code. An interpreter translates one statement, executes it, then moves to the next. It produces no object code, so the source code and interpreter are needed every time the program runs, and execution is slower. Errors are only discovered when the faulty line is reached, which may be after earlier statements have already changed files or data. However, interpreters give immediate feedback, which makes them useful during development and debugging. C# combines both ideas: source code is compiled to an intermediate language, which the .NET runtime compiles to machine code just in time.

Variables, identifiers and constants

A variable is a named location in memory that stores a single value which can change while the program is running. Its name is called its identifier. Most languages, including C#, require variables to be declared before use: the declaration states the data type and identifier so that the right amount of memory can be reserved, and it may also give an initial value. Assignment, using a single equals sign, replaces the current value with a new one. Identifiers must follow the language's rules: in C# they begin with a letter or underscore, contain only letters, digits and underscores, cannot contain spaces, cannot be reserved keywords and are case-sensitive, so total and Total are different variables. Conventions are not enforced by the compiler but should be followed so that code is readable: use meaningful names such as ticketPrice rather than tp, and write variables in camelCase. A constant is a named value fixed when the program is written, declared in C# with const. Using a constant for a value such as a VAT rate or a maximum group size means it cannot be changed accidentally while the program runs, its purpose is clear from its name, and if it ever needs updating it is changed in one place only.

Choosing data types

A data type defines what kind of value a variable can hold, how much memory it needs and which operations can be performed on it. C# uses int for whole numbers such as counts and ages, and double for real numbers such as measurements and averages. For money, decimal is better than double because it stores decimal fractions exactly, avoiding small binary rounding errors. A char holds exactly one character in single quotes, a string holds any sequence of characters in double quotes, and a bool holds only true or false. The choice should match how the data will be used, not just what it looks like. A telephone number or a membership code made of digits should be a string, because it may start with a zero, may contain spaces and will never be used in arithmetic. A yes/no fact, such as whether a fee has been paid, should be Boolean because there are only two possible states. Choosing the right type saves memory, prevents meaningless operations and lets the compiler detect mistakes, such as trying to store text in an integer. Converting between types is called casting: (int)7.9 truncates to 7, while (double)total forces real rather than integer division.

Operators, expressions and conditions

Arithmetic operators in C# are +, -, *, / and %. When both operands of / are integers, C# performs integer division and discards the remainder, so 17 / 5 gives 3; if either operand is real, the result is real, so 17.0 / 5 gives 3.4. The modulus operator % gives the remainder, so 17 % 5 gives 2. Together, DIV and MOD split a quantity into whole units and what is left, for example converting minutes into hours and minutes, and n % 2 == 0 tests whether a number is even. Shorthand operators include ++ to add one and += to add a value to a variable. Relational operators (==, !=, <, <=, >, >=) compare two values and always produce a Boolean result. Boolean operators combine conditions: && (AND) is true only when both sides are true, || (OR) is true when either or both are true, and ! (NOT) reverses a value. To evaluate a condition by hand, substitute the values, work out any brackets first, replace each comparison with true or false, then apply the Boolean operators. Brackets make the intended order clear, since && is evaluated before ||. Watch for the classic slip of writing = (assignment) when == (comparison) is meant.

T Key terms from the mark schemes

Data type
A classification of data that determines the values a variable can hold, the memory it needs and the operations that can be performed on it.

Variable
A named location in memory that stores a single value which can change while the program runs.
`Credited: named storage location`

Constant
A named value that is set when the program is written and cannot change while the program runs.

Identifier
The name given to a variable, constant or subroutine so that it can be referred to in a program.
`Credited: variable name`

Syntax
The set of rules that define how valid statements must be written in a programming language.
`Credited: grammar of the language`

Integrated development environment (IDE)
A software application that provides the tools needed to develop programs, such as an editor, translator and debugging tools, in one place.
`Credited: IDE`

Assembler
A translator that converts assembly language into machine code, each mnemonic usually becoming one machine code instruction.
`Credited: one-to-one translation`

Compiler
A translator that converts the whole of a high-level program into object code before it is executed, reporting any errors found.

Interpreter
A translator that translates and executes a high-level program one statement at a time, producing no object code.

Modulus
An operator (% in C#, MOD in pseudocode) that gives the remainder after integer division.
`Credited: MOD`

Integer division
Division that gives only the whole-number part of the result, discarding any remainder (DIV).
`Credited: DIV`

Casting
Converting a value from one data type to another, e.g. (double)total or (int)7.9.
`Credited: type conversion`

C C# example

`Declaring, assigning and constants`

```
const double VAT_RATE = 0.2;
string item = "Hoodie";
int quantity = 3;
double price = 24.50;
bool member = true;
char size = 'M';
double total = quantity * price;        // 73.5
if (member) total = total * 0.9;        // 10% member discount
total = total + total * VAT_RATE;
Console.WriteLine($"{quantity} x {item} ({size}): £{total:F2}");
// Output: 3 x Hoodie (M): £79.38
```

Exam focus

Questions on this lesson use the command words state, describe and explain. To describe IDE features, name the feature and say how it helps, for example 'syntax highlighting colours keywords so the structure is easier to read' rather than just listing tools; three features are typically worth one or two marks each. For translators, mark schemes credit clear contrasts: a compiler translates the whole program before execution and produces an executable, while an interpreter translates and executes one statement at a time and produces no object code; an assembler translates assembly language with a one-to-one mapping. Avoid saying an interpreter 'has no errors' or that a compiler 'runs the program line by line'. When asked to define a variable, include that it is a named memory location whose value can change during execution. For data type questions, give the type and, when asked, a reason tied to how the data is used: phone numbers are strings because of the leading zero and no arithmetic. When evaluating expressions, show the substituted values, take care with integer division and MOD, and give Boolean results as true or false. Common mistakes are using a real type for a count, and confusing = with ==.

? Check yourself

1. A Belfast software start-up is choosing an IDE for its new apprentices. Describe three features of an IDE that would help the apprentices write correct source code. [6]
2. An apprentice says the C# file they typed 'is the program the CPU runs'. State what is meant by translation, and why it is needed before the program can run. [2]
3. Explain two differences between a compiler and an interpreter. [4]

Answers: 1. Any three features, each with description of how it helps (2 marks each): syntax highlighting / colour coding (1) makes keywords and structure easier to read (1) / automatic indentation (1) shows which statements belong to which block (1) / syntax checking as you type (1) underlines errors such as a missing semicolon so they are fixed before compiling (1) / code completion or IntelliSense (1) suggests identifiers, reducing spelling mistakes (1) / preferences or auto-formatting (1) keep a team's layout consistent (1) / debugging tools such as breakpoints and stepping (1) let the programmer inspect variables to locate errors (1) 2. Converting a program from the language it is written in (source code, e.g. C#) (1) into machine code / object code that the processor can execute, because the CPU cannot run high-level source code directly (1) 3. Any two differences, two marks each: a compiler translates the whole program before execution (1) whereas an interpreter translates and executes one statement at a time (1) / a compiler produces an executable/object code (1) whereas an interpreter produces no object code so it is needed each time (1) / a compiler reports all errors after translation (1) whereas an interpreter stops at the first error encountered (1) / compiled code runs faster (1) because an interpreter retranslates each statement every time it runs (1)